

FitPlanner

Aplicacion de Planificacion de Rutinas de Ejercicio

Objetivos del Proyecto

- Solucionar la falta de estructura en entrenamientos personales.
- Proporcionar una base de datos de ejercicios con guía visual (GIFs).
- Permitir la personalización total de rutinas y días de entrenamiento.
- Garantizar una experiencia de usuario fluida con temas claros/oscuros.

Stack Tecnológico

React 19

Frontend SPA con Vite y Axios.

.NET 10

Backend API robusto y tipado.

PostgreSQL 17

Persistencia de datos relacionales.

Docker

Contenedores para despliegue consistente.

Cloudflare

Gestión de DNS y seguridad SSL.

Nginx

Proxy inverso y servidor de estáticos.

Arquitectura del Sistema

- Frontend: Cliente React consumiendo REST API.
- Backend: Arquitectura en capas (Controller -> Service -> Repository).
- Seguridad: Autenticacion basada en Tokens JWT.
- Infraestructura: Contenedores orquestados via Docker Compose.
- Flujo: Usuario -> Cloudflare -> Nginx -> Backend -> PostgreSQL.

Modelo de Datos

- Usuarios: Gestion de perfiles via ASP.NET Identity.
- Ejercicios: Catalogo con tags de musculos y GIFs.
- Rutinas: Definiciones personalizables por el usuario.
- Planificacion: Relacion M:N entre Rutinas y Dias de la Semana.
- Integridad: Uso de claves foraneas y cascada en EF Core.

Implementacion: Seguridad

```
// Generacion de Token JWT en AuthController.cs
var claims = new[] {
    new Claim(ClaimTypes.NameIdentifier, user.Id),
    new Claim(ClaimTypes.Name, $"{user.FirstName} {user.LastName}")
};
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
var token = new JwtSecurityToken(
    issuer: jwtIssuer, claims: claims,
    expires: DateTime.UtcNow.AddMinutes(60),
    signingCredentials: creds
);
return new JwtSecurityTokenHandler().WriteToken(token);
```

Implementacion: Logica de Rutinas

```
// Creacion de Rutina con asignacion de Usuario
[Authorize]
[HttpPost]
public async Task<IActionResult> Create([FromBody] RoutineDto dto) {
    var routine = new Routine {
        Name = dto.Name,
        UserId = User.FindFirstValue(ClaimTypes.NameIdentifier)
    };
    _context.Routines.Add(routine);
    await _context.SaveChangesAsync();
    return Ok(routine);
}
```

Implementacion: Frontend API

```
// Interceptor de Axios para manejo de JWT
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) window.location.href = "/login";
    return Promise.reject(err);
  }
);
```


Implementacion: UI y Estado

- Uso de Hooks (useState, useEffect) para gestion de datos.
- Renderizado condicional para temas (Clase .light-theme en body).
- Integracion de Drag and Drop para el planificador semanal.
- Uso de Portals para modales, evitando clipping de CSS.

Despliegue y DevOps

```
# docker-compose.yml (Simplificado)
services:
  db:
    image: postgres:17-alpine
  backend:
    build: .
    depends_on: [db]
  frontend:
    build: ./frontend
    ports: ["80:80"]
    # Nginx hace el proxy de /api -> backend:8080
```

Desafios y Soluciones

- CORS: Solucionado mediante configuracion dinamica en Program.cs.
- Zonas Horarias: Uso de DateTimeKind.Utc para persistencia en PostgreSQL.
- Performance: Cache estatica para el catalogo de ejercicios.
- Sincronizacion: Implementacion de SemaphoreSlim en servicios asincronos.

Conclusiones y Futuro

- Objetivo cumplido: Aplicacion funcional, escalable y desplegable.
- Aprendizaje: Dominio de .NET 10, React y orquestacion con Docker.
- Futuro: Implementacion de notificaciones push y seguimiento de pesos.
- Futuro: Version movil nativa utilizando React Native.